

EXPERIMENT 2

Write an OpenMP program that divides the iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread.

For example, if there are two threads and four iterations, the output might be the following:

a. Thread 0: Iterations 0 – 1 b. Thread 1: Iterations 2 – 3

CODE:

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int num_iterations;

    printf("Enter the number of iterations: ");
    scanf("%d", &num_iterations);

    #pragma omp parallel
    {
        #pragma omp for schedule(static,2)
        for (int i = 0; i < num_iterations; i++)
        {
            printf("Thread %d: Iteration %d\n",
                   omp_get_thread_num(), i);
        }
    }

    return 0;
}
```

OUTPUT:

ubuntu@ubuntu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~\$ gcc -fopenmp iterations.c

ubuntu@ubuntu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~\$./a.out

Enter the number of iterations: 6

Thread 1: Iteration 2

Thread 1: Iteration 3

Thread 2: Iteration 4

Thread 2: Iteration 5

Thread 0: Iteration 0

Thread 0: Iteration 1

EXPERIMENT 3

Write a OpenMP program to calculate n Fibonacci numbers using tasks.

CODE:

```
#include <stdio.h>
#include <omp.h>

int fib(int n)
{
    int i, j;

    if (n < 2)
        return n;
    else
    {
        #pragma omp task shared(i) firstprivate(n)
        i = fib(n - 1);

        #pragma omp task shared(j) firstprivate(n)
        j = fib(n - 2);

        #pragma omp taskwait

        return i + j;
    }
}

int main()
{
    int n = 10;

    omp_set_dynamic(0);
    omp_set_num_threads(4);

    #pragma omp parallel shared(n)
    {
        #pragma omp single
        printf("fib(%d) = %d\n", n, fib(n));
    }

    return 0;
}
```

OUTPUT:

ubuntu@ubuntu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~\$ gcc -fopenmp fibonacci.c

```
ubuntu@ubuntu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ ./a.out
```

```
fib(10) = 55
```